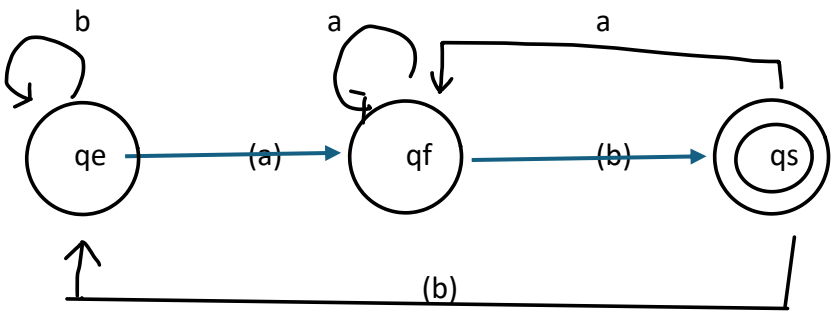
Ques num 1

1(a)

state	input(a)	input(b)	input(c)
q0	q1	-	-
q1	q2	q1	-
q2	-	q2	q3
*q3	-	-	-

Inputs Accepted: The automaton accepts strings consisting of a, b, and c that lead to the acceptance state q3q3.

1(b):



Inputs Accepted: The automaton accepts strings consisting of a and b where the number of a's in the string is congruent to 2 modulo 3. In set builder notation:

L={ω∈{a,b}∗|The number of a's in ω modulo 3 equals 2}L={ω∈{a,b}∗|The number of a's in ω modulo 3 equals 2}

Ques number 2

2(a):

state	input(l)	input(n)	input(+-*/)	input(.)
S	I	L	-	-
*I	I	-	-	-
*L	-	L	O	D

O	-	C	-	-
*C	-	C	O	D
D	-	L,C	-	-

2(B):

This is nondeterministic Finite Automata as at D state, there are L,C transitions for a character in the alphabet.

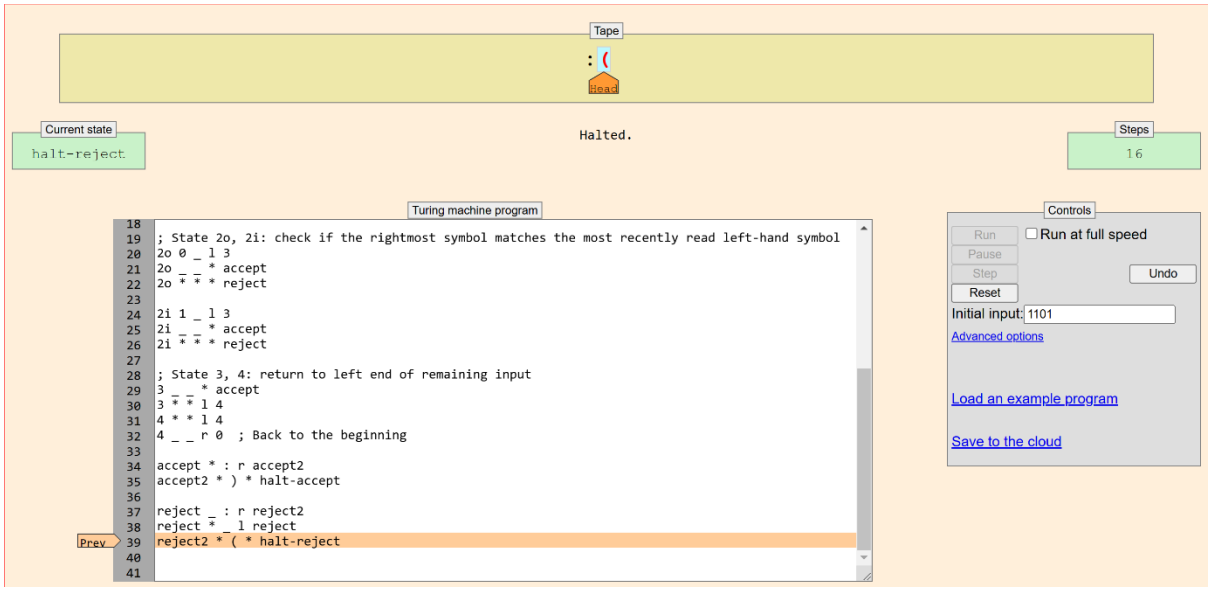
2(c):

state	input(l)	input(n)	input(+-*/)	input(.)
→ S	I	L	-	-
*I	I	-	-	-
~~*L~~	~~-~~	~~L~~	~~O~~	~~D~~
O	-	C	-	-
~~*C~~	~~-~~	~~C~~	~~O~~	~~D~~
D	-	L,C	-	-
L,C	-	L,C	O	D

No transitions takes us from some states to L or C, we can remove them.

Ques num 3

3(a) : a.16 steps



b. 15 steps:

0 0 _ r 1o

0 1 _ r 1i

0 _ _ * accept ; Empty input

; State 1o, 1i: find the rightmost symbol

1o _ _ l 2o

1o * * r 1o

1i _ _ l 2i

1i * * r 1i

; State 2o, 2i: check if the rightmost symbol matches the most recently read left-hand symbol

2o 0 _ l 3

2o _ _ * accept

2o * * * reject

2i 1 _ l 3

2i _ _ * accept

2i * * * reject

; State 3, 4: return to left end of remaining input

3 _ _ * accept

3 * * l 4

4 * * l 4

4 _ _ r 0 ; Back to the beginning

accept * : r accept2

accept2 *) * halt-accept

reject _ : r reject2

reject * _ l reject

reject2 * (* halt-reject

c. 52 steps.

; State 0: read the leftmost symbol

```
0 0 _ r 1o

0 1 _ r 1i

0 _ _ * accept     ; Empty input

; State 1o, 1i: find the rightmost symbol

1o _ _ l 2o

1o * * r 1o

1i _ _ l 2i

1i * * r 1i

; State 2o, 2i: check if the rightmost symbol matches the most recently read left-hand symbol

2o 0 _ l 3

2o _ _ * accept

2o * * * reject

2i 1 _ l 3

2i _ _ * accept

2i * * * reject

; State 3, 4: return to left end of remaining input

3 _ _ * accept

3 * * l 4

4 * * l 4

4 _ _ r 0  ; Back to the beginning

accept * : r accept2

accept2 * ) * halt-accept

reject _ : r reject2

reject * _ l reject

reject2 * ( * halt-reject
```

3(b):

The machine performs binary inversion on the input string. It reads the input from right to left, inverts the bits (0 becomes 1 and 1 becomes 0), and halts when it reaches the end of the input.

Ques no 4

4(a): (took help from ai)
This language requires a machine to ensure that the number of as matches the number of bs. A **finite automaton** (DFA or NFA) cannot do this because it lacks memory to count how many as appear before bs start.

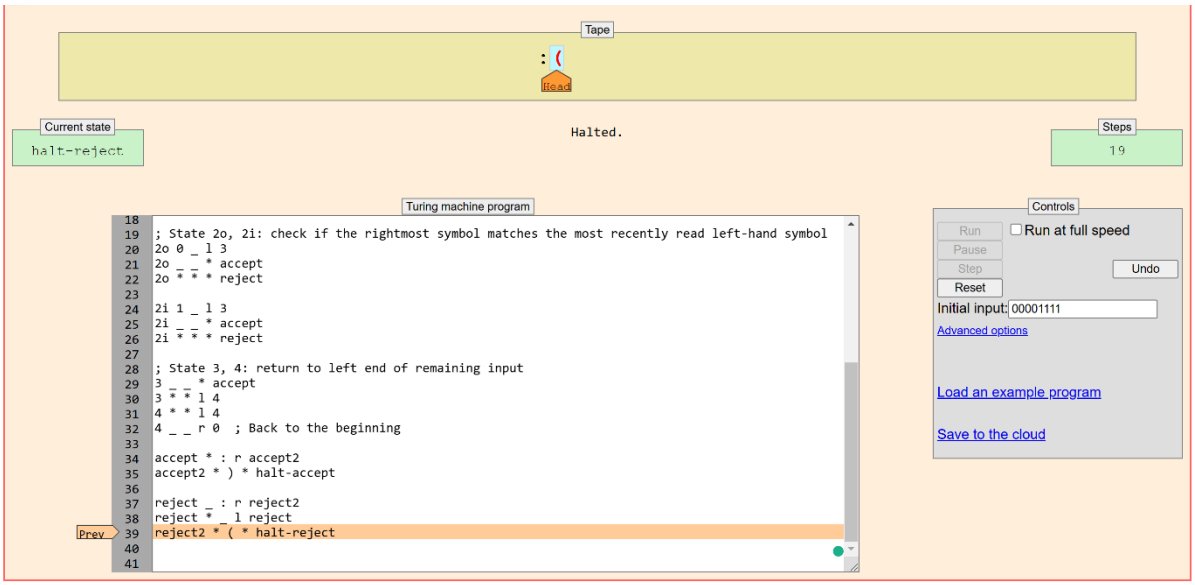
The best type of automaton for this language would be a **pushdown automaton (PDA)** because it can use a stack to keep track of the number of as and match them with bs. However, even a PDA can only recognize **context-free** languages, and this language is **context-sensitive** (not context-free).

A **Turing machine (TM)** is needed to fully recognize and process this language because it has unlimited memory and can read/write on the tape.

Answer: A finite automaton cannot recognize this language because it requires counting, which is beyond its capability. A PDA can handle counting but is still not powerful enough to recognize all cases. A Turing machine is the best choice because it has an infinite tape for storing and processing input.

4(B): HERE, a = 0 and b = 1.
a^ib^i = o^i1^I and i = 4

there are 19 steps.



code:
; State 0: read the leftmost symbol

0 0 _ r 1o

0 1 _ r 1i

```
0 _ _ * accept     ; Empty input

; State 1o, 1i: find the rightmost symbol

1o _ _ l 2o

1o * * r 1o

1i _ _ l 2i

1i * * r 1i

; State 2o, 2i: check if the rightmost symbol matches the most recently read left-hand symbol

2o 0 _ l 3

2o _ _ * accept

2o * * * reject

2i 1 _ l 3

2i _ _ * accept

2i * * * reject

; State 3, 4: return to left end of remaining input

3 _ _ * accept

3 * * l 4

4 * * l 4

4 _ _ r 0  ; Back to the beginning

accept * : r accept2

accept2 * ) * halt-accept

reject _ : r reject2

reject * _ l reject

reject2 * ( * halt-reject
```